

Objetivos da aula:

- Compreender a linguagem PL/pgSQL e sua relação com o PostgreSQL;
- Criar e utilizar funções armazenadas, identificando seus tipos de retorno;
- Criar e executar procedures armazenadas, utilizando parâmetros IN e OUT;
- Diferenciar corretamente funções e procedures, escolhendo a abordagem adequada para cada cenário;
- Interpretar e corrigir erros comuns relacionados à programação no banco de dados.

Índice:

- i. Introdução a Procedures e Funções Armazenadas
- ii. Linguagem PL/pgSQL
- iii. Funções Armazenadas no PostgreSQL
- iv. Procedures Armazenadas no PostgreSQL

i. Introdução a Procedures e Funções Armazenadas**1.1. Introdução**

À medida que aplicações que utilizam bancos de dados relacionais se tornam mais complexas, surge a necessidade de centralizar regras de negócio, reduzir repetição de código e executar operações diretamente no SGBD de forma controlada e eficiente.

Nesse contexto, os bancos de dados oferecem mecanismos de **programação interna**, conhecidos como **procedures** e **funções armazenadas** (*stored procedures* e *stored functions*).

No PostgreSQL, esses recursos permitem escrever código procedural que é armazenado e executado diretamente no servidor de banco de dados.

1.2. O que são procedures e funções armazenadas**1.2.1 Funções armazenadas (Functions)**

Uma função armazenada é um bloco de código que:

- recebe zero ou mais parâmetros;
- executa instruções SQL e lógicas;
- **retorna obrigatoriamente um valor** (simples ou composto);
- pode ser utilizada em expressões SQL (SELECT, WHERE, JOIN etc.).

Conceitualmente:

Uma função representa uma operação que produz um resultado e pode ser usada como parte de uma consulta.

1.2.2 Procedures armazenadas (Procedures)

Uma procedure armazenada é um bloco de código que:

- recebe parâmetros de entrada e/ou saída;
- executa um conjunto de comandos SQL;
- **não retorna valor diretamente;**
- é invocada por meio do comando CALL;
- é indicada para executar **processos e operações administrativas ou transacionais**.

Conceitualmente:

Uma procedure representa um procedimento que executa ações no banco de dados.

1.3. Motivação para o uso

Procedures e funções armazenadas são utilizadas para:

- centralizar regras de negócio no banco;
- reduzir a quantidade de lógica na aplicação cliente;
- reutilizar código SQL e procedural;
- melhorar a organização e manutenção do sistema;
- reduzir tráfego entre aplicação e banco.

1.4. Considerações Importantes

Alguns pontos relevantes:

- Código em procedures e funções é executado **no servidor**, não na aplicação;
- Erros e exceções devem ser tratados explicitamente;
- O uso excessivo pode dificultar manutenção se não houver boa documentação;
- Alterações exigem cuidados com dependências.

Esses aspectos reforçam a necessidade de **uso consciente e planejado** desses recursos.

ii. Linguagem PL/pgSQL

2.1. Introdução ao PL/pgSQL

O PL/pgSQL (*Procedural Language/PostgreSQL*) é a principal linguagem procedural do PostgreSQL.

Ela estende o SQL tradicional com recursos típicos de linguagens de programação, permitindo:

- uso de variáveis;
- estruturas de decisão;
- estruturas de repetição;
- tratamento de exceções;
- execução de múltiplos comandos SQL de forma controlada.

Procedures e funções armazenadas no PostgreSQL são, em geral, escritas em PL/pgSQL.

2.2. Estrutura básica de um bloco PL/pgSQL

Todo código PL/pgSQL é organizado em blocos, delimitados por BEGIN e END.

```
DECLARE
    -- declaração de variáveis
BEGIN
    -- comandos
END;
```

- A seção DECLARE é **opcional**;
- A seção BEGIN ... END é **obrigatória**.

2.3. Bloco anônimo (DO)

O comando BEGIN ... END sozinho:

- não é SQL
- não é uma transação
- não é uma função
- não é uma procedure

Portanto, o PostgreSQL retornará **erro de sintaxe**.

Para executar PL/pgSQL diretamente, é necessário usar o comando **DO**, que define um **bloco anônimo**.

DO \$\$... \$\$ → executa um bloco PL/pgSQL anônimo

Exemplo:

```
DO $$ -- inicio do bloco
BEGIN
    RAISE NOTICE 'Olá!';
END;
$$ LANGUAGE plpgsql; -- fim do bloco
```

Saída:

```
NOTICE: Olá!
DO
```

A instrução LANGUAGE plpgsql informa ao PostgreSQL em qual linguagem o corpo do bloco deve ser interpretado e executado. Sem essa informação, o PostgreSQL não sabe como interpretar o código entre \$\$... \$\$.

2.4. Declaração de Variáveis

Variáveis devem ser declaradas na seção DECLARE.

Sintaxe básica

```
nome_variavel tipoDeDado [:= valor_inicial];
```

Exemplos

```
DECLARE
    total INTEGER;
    media NUMERIC(10,2);
    nome_pessoa VARCHAR(100);
Com inicialização:
DECLARE
    contador INTEGER := 0;
```

Os tipos de dados são aqueles suportados pelo PostgreSQL.

A atribuição pode ser feita com:

```
contador := 10;
```

Ou a partir de uma consulta SQL:

```
DO $$
DECLARE
    jovem DATE;
    velho DATE;
BEGIN
    SELECT MIN(nascimento), MAX(nascimento)
    INTO velho, jovem
    FROM pessoa;

    RAISE NOTICE 'Jovem: %, velho: %', jovem, velho;
END;
$$ LANGUAGE plpgsql;
```

Saída:

```
NOTICE: Jovem: 2009-12-14,
velho: 1940-01-01
DO
```

2.5. Estruturas Condicionais

2.5.1 IF / ELSIF / ELSE

```
IF condição THEN
    comandos;
ELSIF outra_condição THEN
    comandos;
ELSE
    comandos;
END IF;
```

Exemplo:

```
DO $$
DECLARE
    -- altere a idade para testar
```

Saída:

```
NOTICE: Resultado: Melhor idade
DO
```

```
idade INTEGER = 75;
status VARCHAR(20);
BEGIN
    IF idade < 18 THEN
        status := 'Novo';
    ELSIF idade < 65 THEN
        status := 'Normal';
    ELSE
        status := 'Melhor idade';
    END IF;
    RAISE NOTICE 'Resultado: %', status;
END;
$$ LANGUAGE plpgsql;
```

2.5.2 CASE

```
DO $$
DECLARE
    -- altere o valor para testar
    valor INTEGER = 0;
    resultado VARCHAR(20);
BEGIN
    CASE
        WHEN valor < 0 THEN
            resultado := 'Negativo';
        WHEN valor = 0 THEN
            resultado := 'Zero';
        ELSE
            resultado := 'Positivo';
    END CASE;
    RAISE NOTICE 'Resultado: %', resultado;
END;
$$ LANGUAGE plpgsql;
```

Saída:

NOTICE: Resultado: Zero
DO

2.6. Estruturas de repetição

2.6.1 LOOP

```
LOOP
    comandos;
    EXIT WHEN condição;
END LOOP;
```

Exemplo:

```
DO $$
DECLARE
    contador INTEGER = 1;
BEGIN
    LOOP
        contador := contador + 2;
        RAISE NOTICE 'Valor: %', contador;
        EXIT WHEN contador > 10;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Saída:

```
NOTICE: Valor: 3
NOTICE: Valor: 5
NOTICE: Valor: 7
NOTICE: Valor: 9
NOTICE: Valor: 11
DO
```

2.6.2 WHILE

```
    WHILE condição LOOP
        comandos;
    END LOOP;
```

Exemplo:

```
DO $$
DECLARE
    contador INTEGER = 0;
BEGIN
    WHILE contador <= 5 LOOP
        contador := contador + 1;
        RAISE NOTICE 'Valor: %', contador;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Saída:

```
NOTICE: Valor: 1
NOTICE: Valor: 2
NOTICE: Valor: 3
NOTICE: Valor: 4
NOTICE: Valor: 5
NOTICE: Valor: 6
DO
```

2.6.3 FOR (numérico)

```
    FOR i IN 1..5 LOOP
        comandos;
    END LOOP;
```

Exemplo:

```
DO $$
DECLARE
    i INTEGER;
BEGIN
```

Saída:

```
NOTICE: Valor: 1
NOTICE: Valor: 2
NOTICE: Valor: 3
```

```
FOR i IN 1..5 LOOP
    RAISE NOTICE 'Valor: %', i;
END LOOP;
END;
$$ LANGUAGE plpgsql;
```

```
NOTICE: Valor: 4
NOTICE: Valor: 5
DO
```

2.6.4 FOR com consulta SQL

```
DO $$
DECLARE
    registro RECORD;
BEGIN
    FOR registro IN
        SELECT nome FROM pessoa LIMIT 5
    LOOP
        RAISE NOTICE '%', registro.nome;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Saída:

```
NOTICE: Elaine Oliveira
NOTICE: César Machado
NOTICE: Tatiane Fernandes
NOTICE: Cristina Cury
NOTICE: Gumercindo Carvalho
DO
```

Esse tipo de laço é muito utilizado para percorrer resultados de consultas.

2.7. Mensagens e depuração (RAISE)

O comando RAISE é usado para exibir mensagens:

- NOTICE
- WARNING
- EXCEPTION

Exemplo de erro:

```
DO $$
BEGIN
    RAISE NOTICE 'Olá!';
    RAISE WARNING 'Aviso!';
    RAISE EXCEPTION 'Valor inválido!';
END;
$$ LANGUAGE plpgsql;
```

Saída:

```
NOTICE: Olá!
WARNING: Aviso!

ERROR: Valor inválido!
CONTEXT: função PL/pgSQL
inline_code_block linha 5 em
RAISE
SQL state: P0001
```

2.8. Tratamento de Exceções

O PL/pgSQL permite tratar erros com EXCEPTION.

```
DO $$
DECLARE
    a INTEGER = 10;
    b INTEGER = 0;
    r INTEGER;
BEGIN
    r = a / b;
    RAISE NOTICE 'Resultado: %', r;

    EXCEPTION
        WHEN division_by_zero THEN
            RAISE NOTICE 'Divisão por zero não
permitida';
END;
$$ LANGUAGE plpgsql;
```

Saída:

```
NOTICE: Divisão por zero não
permitida
DO
```

2.9. Uso de Comandos SQL no PL/pgSQL

Dentro de um bloco PL/pgSQL, podem ser utilizados:

- SELECT ... INTO
- INSERT
- UPDATE
- DELETE

2.10. Boas Práticas em PL/pgSQL

- Declarar apenas as variáveis necessárias;
- Usar nomes claros e significativos;
- Evitar lógica excessivamente complexa;
- Comentar o código;
- Tratar exceções sempre que possível.

iii. Funções Armazenadas no PostgreSQL

3.1. Conceito de função armazenada

Uma função armazenada é um objeto do banco de dados que:

- recebe zero ou mais parâmetros;
- executa comandos SQL e lógica procedural;
- **retorna obrigatoriamente um valor;**
- pode ser utilizada dentro de comandos SQL (SELECT, WHERE, JOIN etc.).

Definição conceitual:

Uma função armazenada representa uma operação que produz um resultado e pode ser utilizada como parte de uma consulta SQL.

3.2. Estrutura básica de uma função

```
CREATE FUNCTION nome_funcao(parâmetros)
RETURNS tipo_retorno
LANGUAGE plpgsql
AS $$
DECLARE
    -- declaração de variáveis
BEGIN
    -- comandos
    RETURN valor;
END;
$$;
```

Componentes principais:

- RETURNS: define o tipo do valor retornado;
- DECLARE: seção opcional para variáveis locais;
- RETURN: obrigatório em funções escalares.

Exemplo: função que retorna o ano atual

```
CREATE OR REPLACE FUNCTION ano_atual()
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN EXTRACT(YEAR FROM CURRENT_DATE);
END;
$$;
```

Uso da função:

```
SELECT ano_atual();
```

3.3. Funções com parâmetros

Funções podem receber parâmetros de entrada.

Exemplo: função que calcula a idade aproximada

```
CREATE OR REPLACE FUNCTION calcula_idade(data_nascimento DATE)
RETURNS INTEGER
LANGUAGE plpgsql
```

```
AS $$  
BEGIN  
    RETURN EXTRACT(YEAR FROM AGE(data_nascimento));  
END;  
$$;
```

Exemplo de uso da função:

```
SELECT nome, calcula_idade(nascimento)  
FROM pessoa  
LIMIT 5;
```

3.4. Funções com Consultas (SELECT INTO)

Funções podem executar consultas e armazenar resultados em variáveis.

Exemplo: quantidade de carros de uma pessoa

```
CREATE OR REPLACE FUNCTION qtd_carros(id INTEGER)  
RETURNS INTEGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    total INTEGER;  
BEGIN  
    SELECT COUNT(*)  
    INTO total  
    FROM carro  
    WHERE id_pessoa = id;  
  
    RETURN total;  
END;  
$$;
```

Exemplo de uso da função:

```
SELECT nome, qtd_carros(id_pessoa)  
FROM pessoa  
LIMIT 5;
```

3.5. Funções que retornam tipos compostos

3.5.1 Retorno de um registro (RECORD)

```
CREATE OR REPLACE FUNCTION pessoa_mais_velha()  
RETURNS RECORD  
LANGUAGE plpgsql
```

```
AS $$
DECLARE
    resultado RECORD;
BEGIN
    SELECT id_pessoa, nome, nascimento
    INTO resultado
    FROM pessoa
    ORDER BY nascimento
    LIMIT 1;

    RETURN resultado;
END;
$$;
```

Uso:

```
SELECT *
FROM pessoa_mais_velha() AS (id_pessoa INTEGER, nome VARCHAR, nascimento
DATE);
```

A função `pessoa_mais_velha()` foi definida com:

```
RETURNS RECORD
```

Ou seja, ela retorna um tipo genérico (RECORD), que:

- **não possui estrutura conhecida** fora do corpo da função;
- não informa automaticamente:
 - nomes das colunas;
 - tipos das colunas;
 - quantidade de colunas.

Portanto, **no momento da consulta**, o PostgreSQL **exige** que o chamador informe **explicitamente** como interpretar o RECORD.

Podemos substituir o `RETURNS RECORD` por `RETURNS TABLE`:

```
CREATE OR REPLACE FUNCTION pessoa_mais_velha()
RETURNS TABLE (
    out_id_pessoa INTEGER,
    out_nome VARCHAR,
    out_nascimento DATE
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
```

```
        SELECT id_pessoa, nome, nascimento
        FROM pessoa
        ORDER BY nascimento
        LIMIT 1;
END;
$$;
```

Uso:

```
SELECT *
FROM pessoa_mais_velha();
```

3.5.2 Retorno de conjunto de linhas (SETOF)

```
CREATE OR REPLACE FUNCTION pessoas_nascidas_apos(p_data DATE)
RETURNS SETOF pessoa
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT *
    FROM pessoa
    WHERE nascimento > p_data;
END;
$$;
```

Uso:

```
SELECT *
FROM pessoas_nascidas_apos('2008-01-01');
```

3.6. Uso de estruturas de controle em funções

Funções podem utilizar:

- IF / ELSE;
- LOOP, FOR, WHILE;
- tratamento de exceções.

Exemplo com condição

```
CREATE OR REPLACE FUNCTION classifica_idade(data_nascimento DATE)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    idade INTEGER;
```

```
BEGIN
    idade := EXTRACT(YEAR FROM AGE(data_nascimento));

    IF idade < 18 THEN
        RETURN 'Menor de idade';
    ELSE
        RETURN 'Maior de idade';
    END IF;
END;
$$;
```

Uso:

```
SELECT nome, nascimento, classifica_idade(nascimento)
FROM pessoa
LIMIT 10;
```

3.7. Tratamento de exceção

A função lançará uma exceção caso o parâmetro `p_data` seja nulo.

`RAISE EXCEPTION`: Interrompe a execução imediatamente e envia uma mensagem para a aplicação ou usuário, em vez de retornar um resultado vazio que poderia causar confusão.

```
CREATE OR REPLACE FUNCTION pessoas_nascidas_apos(p_data DATE)
RETURNS SETOF pessoa
LANGUAGE plpgsql
AS $$
BEGIN
    -- 1. Validação de nulidade
    IF p_data IS NULL THEN
        RAISE EXCEPTION 'O parâmetro de data não pode ser nulo (NULL).';
    END IF;

    -- 2. Retorno da consulta
    RETURN QUERY
    SELECT *
    FROM pessoa
    WHERE nascimento > p_data;
END;
$$;
```

Uso:

```
SELECT *
FROM pessoas_nascidas_apos(NULL);
```

3.8. Restrições e considerações importantes

- Funções **não devem executar comandos transacionais** (COMMIT, ROLLBACK);
- Funções devem ser **determinísticas sempre que possível**;
- Funções podem impactar desempenho se usadas em consultas com muitas linhas;
- Uso excessivo pode dificultar manutenção se não houver documentação.

iv. Procedures Armazenadas no PostgreSQL

4.1. Introdução

No capítulo anterior, foram estudadas as funções armazenadas, que retornam valores e podem ser utilizadas dentro de consultas SQL.

Neste capítulo, o foco passa a ser as procedures armazenadas, que têm um propósito diferente: **executar processos** no banco de dados.

As procedures são especialmente úteis para:

- executar múltiplas operações em sequência;
- implementar regras de negócio procedurais;
- controlar transações;
- realizar tarefas administrativas ou operacionais.

4.2. Conceito de procedure armazenada

Uma procedure armazenada é um objeto do banco de dados que:

- recebe parâmetros de entrada e/ou saída;
- executa um conjunto de comandos SQL e lógicos;
- **não retorna valor diretamente**;
- é invocada por meio do comando CALL;
- pode controlar transações (COMMIT, ROLLBACK).

Definição conceitual:

Uma procedure representa um procedimento que executa ações no banco de dados, em vez de produzir um valor para uma consulta.

4.3. Diferença prática entre funções e procedures

Aspecto	Função	Procedure
Retorno de valor	Obrigatório	Não
Forma de chamada	SELECT funcao()	CALL procedure()
Uso em consultas	Sim	Não

Controle de transações	Não	Sim
Uso típico	Cálculos, consultas	Processos e operações

Essa distinção é essencial para escolher corretamente entre função e procedure.

4.4. Estrutura básica de uma procedure

```
CREATE PROCEDURE nome_procedure(parâmetros)
LANGUAGE plpgsql
AS $$
BEGIN
    -- comandos
END;
$$;
```

Ao contrário das funções:

- não há cláusula RETURNS;
- não existe RETURN de valor.

Procedures são executadas com o comando CALL.

```
CALL nome_procedure();
```

Esse comando apenas executa o bloco definido, sem retornar resultado.

4.5. Procedures com parâmetros

4.5.1 Parâmetros de entrada (IN)

Procedures utilizam IN para indicar parâmetro recebido.

```
CREATE OR REPLACE PROCEDURE atualizar_nome_pessoa(
    IN p_id INTEGER,
    IN p_nome VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE pessoa
    SET nome = p_nome
    WHERE id_pessoa = p_id;
END;
$$;
```

Execução:

```
CALL atualizar_nome_pessoa(10, 'Predro Prado Procedure');
```

4.5.2 Parâmetros de saída (OUT)

Procedures utiliza parâmetros OUT para devolver valores indiretamente.

```
CREATE OR REPLACE PROCEDURE contar_carros(  
    IN p_id INTEGER,  
    OUT total INTEGER  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    SELECT COUNT(*)  
    INTO total  
    FROM carro  
    WHERE id_pessoa = p_id;  
END;  
$$;
```

Execução:

```
DO $$  
DECLARE  
    p_total INTEGER; -- Declaramos a variável que receberá o valor  
BEGIN  
    CALL contar_carros(3, p_total);  
    RAISE NOTICE 'Total: %', p_total;  
END;  
$$ LANGUAGE plpgsql;
```

Não podemos passar o nome de uma variável solta em um comando CALL. O banco tentará interpretar `p_total` como uma coluna de uma tabela e falhará.

Por esse motivo declaramos a variável e fazemos a chamada da procedure dentro de um bloco DO.

4.6. Listar funções e procedures disponíveis no esquema 'public'

Utilizar procedures para **processos**, não para consultas;

Para listar funções e procedures no PostgreSQL podemos realizar consultas SQL nas tabelas de catálogo do sistema. O comando a seguir lista todas as funções e procedures disponíveis inclusive aquelas funções incluídas pelas extensões.

```
SELECT  
    n.nspname AS esquema,  
    p.proname AS nome,  
    CASE p.prokind  
        WHEN 'f' THEN 'Função'
```



```

        WHEN 'p' THEN 'Procedure'
        WHEN 'a' THEN 'Agregada'
        WHEN 'w' THEN 'Window'
    END AS tipo,
    pg_get_function_arguments(p.oid) AS argumentos
FROM pg_proc p
JOIN pg_namespace n ON n.oid = p.pronamespace
WHERE n.nspname = 'public'
ORDER BY tipo, nome;

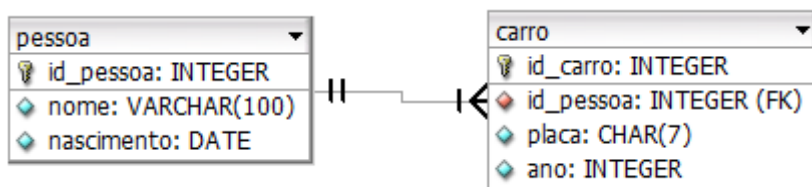
```

4.7. Boas práticas no uso de procedures

- Utilizar procedures para **processos**, não para consultas;
- Evitar lógica excessivamente complexa;
- Nomear procedures com verbos (ex.: inserir_, atualizar_, processar_);
- Documentar parâmetros e comportamento;
- Testar exceções e cenários de erro.

Exercícios

Para a correta execução dos exercícios, utilize os comandos a seguir para criar as tabelas e carregar os registros a partir de arquivos CSV.



Cada arquivo contém 100 mil registros.

Antes de executar os comandos COPY, é necessário **ajustar o caminho dos arquivos** de acordo com a localização dos CSVs no seu computador.

<pre> DROP TABLE IF EXISTS carro, pessoa CASCADE; CREATE TABLE IF NOT EXISTS pessoa (id_pessoa INTEGER PRIMARY KEY, nome VARCHAR(100) NOT NULL, nascimento DATE); </pre>	<pre> COPY pessoa (id_pessoa, nome, nascimento) FROM 'C:/caminho/aula3_pessoa.csv' DELIMITER ',' CSV HEADER; COPY carro (id_carro, placa, ano, id_pessoa) </pre>
--	--

```
CREATE TABLE IF NOT EXISTS carro (  
    id_carro INTEGER PRIMARY KEY,  
    placa CHAR(7) NOT NULL UNIQUE,  
    ano INTEGER,  
    id_pessoa INTEGER NOT NULL,  
    FOREIGN KEY (id_pessoa)  
        REFERENCES pessoa(id_pessoa)  
        ON DELETE CASCADE  
);  
  
FROM 'C:/caminho/aula3_carro.csv'  
DELIMITER ','  
CSV HEADER;
```

Exercício 1 – Bloco anônimo em PL/pgSQL

Escreva um bloco anônimo (DO) em PL/pgSQL que exiba, utilizando RAISE NOTICE, os nomes e idades das 5 primeiras pessoas cadastradas na tabela pessoa.

Resposta:

```
DO $$  
DECLARE  
    registro RECORD;  
BEGIN  
    FOR registro IN  
        SELECT nome, nascimento FROM pessoa LIMIT 5  
    LOOP  
        RAISE NOTICE '% - %', registro.nome, registro.nascimento;  
    END LOOP;  
END;  
$$;
```

Exercício 2 — Uso de variáveis e SELECT INTO

O código a seguir obtém o ano médio dos nascimentos na tabela pessoa:

```
SELECT ROUND(AVG(EXTRACT(YEAR FROM nascimento)))  
FROM pessoa;
```

Crie um bloco PL/pgSQL que determine o ano médio dos nascimentos da tabela pessoa, exibindo os valores com RAISE NOTICE.

Resposta:

```
DO $$  
DECLARE  
    media_ano INTEGER;  
BEGIN  
    SELECT ROUND(AVG(EXTRACT(YEAR FROM nascimento)))
```

```
        INTO media_ano
    FROM pessoa;

    RAISE NOTICE '% é o ano médio dos nascimentos', media_ano;
END;
$$ LANGUAGE plpgsql;
```

Exercício 3 — Função escalar simples

Crie uma função armazenada que recebe o ano e retorne a quantidade de pessoas que nasceram naquele ano.

Resposta:

```
CREATE OR REPLACE FUNCTION total_nascimentos_por_ano(p_ano INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_total INTEGER;
BEGIN
    SELECT COUNT(*)
    INTO v_total
    FROM pessoa
    WHERE EXTRACT(YEAR FROM nascimento) = p_ano;

    RETURN v_total;
END;
$$;
```

Uso:

```
SELECT total_nascimentos_por_ano(2000) AS total;
```

Exercício 4 — Função que retorna conjunto (RETURNS TABLE)

Crie uma função que retorne os dados da pessoa mais jovem cadastrada no banco.

Resposta:

```
CREATE OR REPLACE FUNCTION pessoa_mais_jovem()
RETURNS TABLE (
    id_pessoa INTEGER,
    nome VARCHAR,
    nascimento DATE
)
LANGUAGE plpgsql
```

```
AS $$
BEGIN
    RETURN QUERY
    SELECT p.id_pessoa, p.nome, p.nascimento
    FROM pessoa p
    ORDER BY p.nascimento DESC
    LIMIT 1;
END;
$$;
```

Uso:

```
SELECT * FROM pessoa_mais_jovem();
```

Exercício 5 — Função com estrutura condicional

Crie uma função que receba uma data de nascimento e retorne se a pessoa nasceu em um ano bissexto “é bissexto” ou “não é bissexto”.

Para verificar se um ano é bissexto, precisamos aplicar a regra lógica: o ano deve ser divisível por 4 e não ser divisível por 100, ou ser divisível por 400.

Resposta:

```
CREATE OR REPLACE FUNCTION classifica_bissexto(data_nasc DATE)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    v_ano INTEGER;
BEGIN
    v_ano := EXTRACT(YEAR FROM data_nasc);

    -- Regra do ano bissexto:
    -- (Divisível por 4 E não divisível por 100) OU (Divisível por 400)
    IF (v_ano % 4 = 0 AND v_ano % 100 != 0) OR (v_ano % 400 = 0) THEN
        RETURN v_ano || ' é Bissexto';
    ELSE
        RETURN v_ano || ' não é Bissexto';
    END IF;
END;
$$;
```

Uso:

```
SELECT nome, classifica_bissexto(nascimento)
```

```
FROM pessoa
LIMIT 5;
```

Exercício 6 — Procedure simples com parâmetro IN

Crie uma procedure que atualize a placa de um carro, recebendo o id e a nova placa como parâmetros.

Resposta:

```
CREATE OR REPLACE PROCEDURE atualizar_placa_carro(
    IN p_id INTEGER,
    IN p_placa VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE carro
    SET placa = p_placa
    WHERE id_carro = p_id;
END;
$$;
```

Uso:

```
CALL atualizar_placa_carro(5, 'AAF0A05');
```

Exercício 7 — Procedure com parâmetro OUT

Crie uma procedure que receba o id de uma pessoa e retorne, via parâmetro OUT, o ano do carro mais novo que ela possui.

Resposta:

```
CREATE OR REPLACE PROCEDURE buscar_carro_novo(
    IN p_id INTEGER,
    OUT novo INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT ano
    INTO novo
    FROM carro
    WHERE id_pessoa = p_id
    ORDER BY ano DESC
    LIMIT 1;
```

```
END;  
$$;
```

Uso:

```
DO $$  
DECLARE  
    v_novo INTEGER; -- Declaramos a variável que receberá o valor  
BEGIN  
    CALL buscar_carro_novo(3, v_novo);  
    RAISE NOTICE 'Carro mais novo: %', v_novo;  
END;  
$$ LANGUAGE plpgsql;
```

Exercício 8 — Procedure com controle de exceções

Crie uma procedure que insira uma pessoa e trate erros de forma adequada, exibindo uma mensagem em caso de falha.

Resposta:

```
CREATE OR REPLACE PROCEDURE inserir_pessoa_segura(  
    IN p_id_pessoa INTEGER,  
    IN p_nome VARCHAR,  
    IN p_nascimento DATE  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO pessoa (id_pessoa, nome, nascimento)  
    VALUES (p_id_pessoa, p_nome, p_nascimento);  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'Erro ao inserir pessoa';  
END;  
$$;
```

Uso:

```
CALL inserir_pessoa_segura(10, 'Ana Maria', '1995-06-10');
```